Dual-Pivot Quicksort

LIN Zhanzhi

***Abstract* — This project presents a modern C++17 dual-pivot quicksort library as a practical drop-in alternative to std::sort for in-memory sorting on shared-memory CPUs. It combines Yaroslavskiy's dual-pivot partitioning with median-of-five sampling, mixed insertion sort for small subarrays, Dutch National Flag handling of duplicates, and an introspective heapsort fallback. Adaptive extensions include Timsort-style run detection and merging and counting-sort specializations for 8- and 16-bit integer types. A custom benchmarking framework spanning over 7,800 configurations shows that the implementation matches or slightly exceeds std::sort on random inputs, achieves up to ~19× speedup on structured data, and attains 5.76× speedup on 16 threads for 10 million integers with a custom work-stealing thread pool.**

Keywords* — *Dual-pivot quicksort, adaptive sorting, work-stealing, C++17, performance analysis

I. Introduction

Sorting remains a fundamental systems operation, but existing C++ standard library implementations still rely mainly on single-pivot Introsort despite the practical success of dual-pivot quicksort in Java [1][2]. The core motivation of this project is to determine whether Yaroslavskiy's reduction in scanned elements and memory traffic can translate into real wall-clock speedups in native C++ code, where comparator overhead is already very low. The project therefore targets both algorithm engineering and empirical validation: building a production-quality implementation, benchmarking it rigorously against GCC std::sort, and explaining the observed performance at the hardware level [3].

II. Design and Implementation

- Header-only C++17 library; generic over random-access iterators and usable as a drop in replacement for std::sort.
- Sequential core: dual-pivot three-way partitioning, median-of-five sampling, mixed insertion sort for small subarrays, and introspective heapsort fallback for worst-case safety [4].
- Adaptive paths: Timsort-style run merger for structured inputs and counting-sort specializations for int8_t/int16_t to exploit small key ranges.
- Parallelization: custom work-stealing thread pool with per-worker mutex-protected deques (local LIFO pops, remote FIFO steals) and a sticky-victim policy to reduce steal overhead [5].

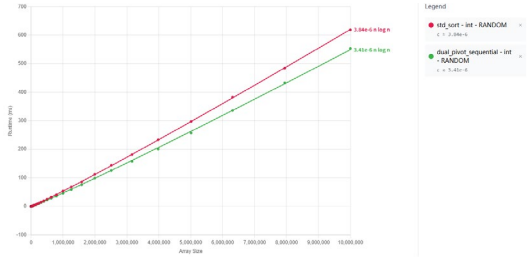

Fig. 1. Runtime comparison of sequential Dual-Pivot Quicksort and std::sort on random 32-bit integers.

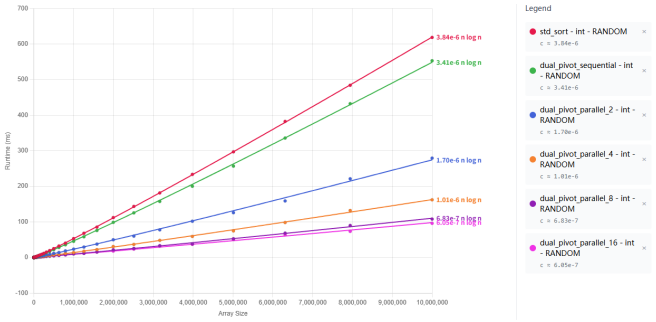

Fig. 2. Runtime on random integers for std::sort and Dual-Pivot Quicksort with 1, 2, 4, 8, and 16 threads.

III. Evalauation and Results

Fig. 1 shows that the sequential dual-pivot implementation is competitive with std::sort on random 32-bit integers, while additional experiments (not shown) demonstrate much larger gains—up to about 19×—on highly structured, run-rich inputs due to the Timsort-style run merger.

Fig. 2 reports the parallel scaling on random integers: Dual-Pivot Quicksort reaches a 5.76× speedup at 16 threads for 10 million elements, but scaling flattens beyond 8 threads. Intel VTune analysis attributes this plateau to the memory wall rather than algorithm design, with most pipeline slots at high thread counts lost to cache-miss stalls, branch mispredictions, and front-end limitations.

IV. Conclusion

This project demonstrates that dual-pivot quicksort can be engineered into a practical, modern C++ sorting library with strong real-world performance. Its main contributions are a robust generic implementation, adaptive optimizations for structured and bounded-range inputs, a custom work-stealing parallel runtime, and a detailed hardware-level evaluation explaining where speedups come from and where they stop. Future work identified in the report includes SIMD vectorization, block-based partitioning inspired by BlockQuicksort, and improved handling of nearly-sorted inputs.

References

[1] C. A. R. Hoare, Quicksort, The Computer Journal, Volume 5, Issue 1, 1962, Pages 10–16, https://doi.org/10.1093/comjnl/5.1.10.

[2] V. Yaroslavskiy, "Dual-Pivot Quicksort," 2009.

[3] D. R. Musser, "Introspective sorting and selection algorithms," Software: Practice and Experience, vol. 27, no. 8, pp. 983–993, Aug. 1997, doi: 10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#.

[4] M. E. Nebel, S. Wild, and C. Martínez, "Analysis of pivot sampling in dual-pivot quicksort: A holistic analysis of Yaroslavskiy's partitioning scheme," Algorithmica, vol. 80, no. 2, pp. 790–848, Jun. 2016.

[5] R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," Journal of the ACM, vol. 46, no. 5, pp. 720–748, Sep. 1999.